#1 A comparison method to decide which shop is preferable in our search. It takes both shops (current node's and the nearest neighbor found so far) as well as their distances.
#2 If `shop` is `null`, or it doesn't have all the items in the purchase list (`boughtItems`), then we return 1 to let the caller know that current nearest neighbor is still the best solution.
#3 Otherwise, if we haven't yet stored a NN, return `-1`: current node is better (than nothing!).
#4 If the distance of current node is not worse than what we had found, we can compare the two shops using a heuristic and see which one is better. Here this heuristic encapsulates any business logic, for instance if a shop has lower shipment costs, if they sell the goods at lower price, or any other domain-specific condition.
#5 If everything else fails, we just stick with what we have currently saved as nearest neighbor.
#6 An example of a call to the (filtered) nearest neighbor method, searching for the most convenient shop that sells all 3 items in the list.

So, now the hearth of the business logic lies into this method, that in turn will decide which shops are filtered in and how we choose our best option.

There are a couple of edge cases that we should always address in this function:

- If current node is filtered out, always choose current NN.
- If the best solution found so far is empty (i.e. `nn == null`), then current node should always be chosen, unless it is filtered out (at point 1).